

MANUAL DE IMPLEMENTAÇÃO

Rede LoRaWAN Completa com PKLoRaWAN

Gateway Monocanal ESP32 + RFM95 | End Device Sensor LDR

ChirpStack v4 (Docker) | Banda AU915 (916.8 MHz)

Versão 1.0 | Junho 2026

Sumário

1. Introdução e Visão Geral da Infraestrutura	3
1.1 Especificações de Operação	3
1.2 Materiais e Componentes de Hardware	3
1.3 Visão Geral da Arquitetura de Comunicação	4
1.4 Arquitetura de 5 Camadas do End Device (TpM)	4
2. Instalação de Ferramentas e Ambiente de Software	5
2.1 Instalação do Docker e Docker Compose	5
2.2 Instalação e Configuração da Arduino IDE	5
2.2.1 Download e Instalação	6
2.2.2 Adição da URL de Placas ESP32	6
2.2.3 Instalação do Core ESP32	6
2.3 Instalação das Bibliotecas Arduino	6
3. Network Server - ChirpStack v4 via Docker	7
3.1 Clonagem do Repositório	7
3.2 Configuração do docker-compose.yml	7
3.3 Configuração Regional AU915	9
3.4 Inicialização dos Serviços	9
4. Configuração e Firmware do Gateway Monocanal ESP32	10
4.1 Wiring - Conexão RFM95 ao ESP32 (DevKit v1)	10
4.2 Configuração do Firmware do Gateway	10
4.2.1 Arquivo loraModem.h - Pinagem e Frequências	10
4.2.2 Arquivo configGway.h - Configurações Principais	11
4.2.3 Arquivo configNode.h - WiFi e Identificação	12
4.2.4 Arquivo _utils.ino - Correção de Tipo de Dado	12
4.2.5 Arquivo ESP_sc_gway_pk.ino - Reconexão WiFi	12
4.3 Upload do Firmware do Gateway	12
5. Configuração e Firmware do End Device ESP32	13
5.1 Wiring - Conexões do End Device	13
5.2 Configuração da Biblioteca LMIC	13
5.2.1 Arquivo lmic_project_config.h	13
5.2.2 Correção no Arquivo hal.cpp da Biblioteca LMIC	14
5.3 Configuração do Canal AU915 no Firmware	14
5.4 Credenciais ABP	15
5.5 Arquitetura em Camadas do Payload	15
5.5.1 Camada PHY (1_PHY.ino)	15
5.5.2 Camada MAC (2_MAC.ino)	15
5.5.3 Camada NET (3_NET.ino)	15
5.5.4 Camada TRANSP (4_TRANSP.ino)	16
5.5.5 Camada APP (5_APP.ino)	16
5.6 Upload do Firmware do End Device	16

6. Comissionamento no ChirpStack e Validação da Rede	16
6.1 Criação do Gateway Profile	16
6.2 Registro do Gateway	17
6.3 Criação do Device Profile	17
6.4 Registro do End Device	17
6.5 Validação do Sistema	17
6.6 Verificação do Payload no ChirpStack	18
7. Troubleshooting	18
7.1 Comandos Úteis de Diagnóstico	19

1. Introdução e Visão Geral da Infraestrutura

Este manual descreve todas as configurações e alterações necessárias para implementar uma infraestrutura LoRaWAN funcional baseada no projeto PKLoRaWAN. A solução adota uma arquitetura de três camadas integradas:

- End Device — Nó sensor composto por ESP32 + módulo rádio RFM95 (SX1276), responsável pela coleta de dados do sensor LDR (luminosidade), tensão da bateria, estado de botão e coordenadas GPS, transmitindo via LoRaWAN Classe A em modo ABP (Activation By Personalization) na frequência de 916.8 MHz.
- Gateway Monocanal — Implementado em ESP32 + RFM95, opera com detecção CAD (Channel Activity Detection) multi-SF, suporte a atualização OTA e comunicação com o Network Server via protocolo Semtech UDP.
- Network Server — ChirpStack v4 dockerizado com PostgreSQL (banco de dados), Redis (cache), Mosquitto (broker MQTT), Gateway Bridge e REST API.

1.1 Especificações de Operação

O projeto opera na banda AU915_928 (Austrália / Brasil), utilizando o canal 8 que corresponde a frequência primária de 916.8 MHz com largura de banda de 125 kHz e Spreading Factor 7 (SF7).

Parâmetro	Valor
Banda de Frequência	AU915_928
Frequência Primária	916.8 MHz (Canal 8)
Largura de Banda	125 kHz
Spreading Factor	SF7
Classe LoRaWAN	Classe A
Modo de Ativação	ABP (Activation By Personalization)
Protocolo Gateway	Semtech UDP
Porta UDP	1700
Versão LoRaWAN	1.0.3 / Revision B

1.2 Materiais e Componentes de Hardware

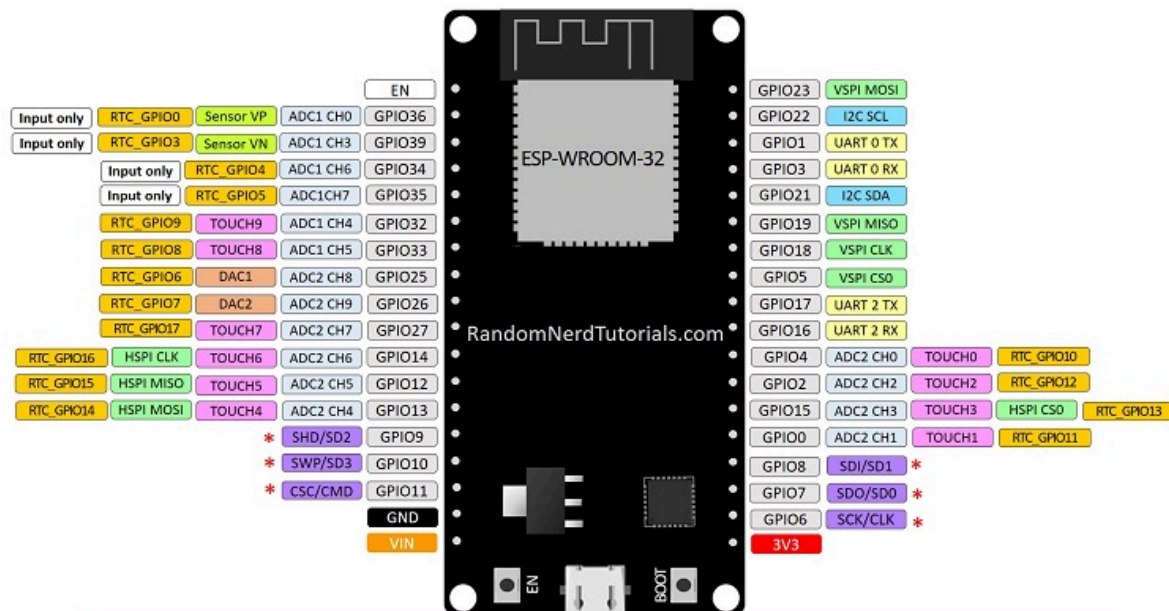
A tabela a seguir lista todos os materiais necessários para a reprodução completa da infraestrutura:

Item	Especificação	Quantidade
Placa Gateway	ESP32 DevKit v1 (30 pinos)	1
Placa End Device	ESP32 DevKit v1 (30 pinos)	1

Item	Especificação	Quantidade
Módulo Rádio Gateway	RFM95W / SX1276 (915 MHz)	1
Modulo Radio End Device	RFM95W / SX1276 (915 MHz)	1
Sensor LDR	Fotoreistor (5mm)	1
Resistor Pull-down	10 kOhms	1
Resistor LED	220 Ohms	3
LED Vermelho	5mm (indicação TX)	1
LED Amarelo	5mm (indicação status)	1
LED Verde	5mm (indicação RX/OK)	1
Botão	Push-button (pull-down)	1
Cabos USB	USB-A para micro-USB	2
Jumpers	Fêmea-fêmea, ~20 unidades	1 conjunto
Protoboard	400 pontos (opcional, para testes)	1

1.3 Imagens de Exemplo

ESP32 DEVKIT V1 – DOIT version with 36 GPIOs



* Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and CSC/CMD, namely, GPIO6 to GPIO11 are connected to the integrated SPI flash integrated on ESP-WROOM-32 and are not recommended for other uses.

Esp32 usada no kit PKLoRa

1.4 Visão Geral da Arquitetura de Comunicação

O fluxo de comunicação segue o padrão LoRaWAN em três camadas independentes. O End Device coleta dados do sensor LDR e monta um pacote MoT (Management Over Tunneling) de 36 bytes seguindo a arquitetura em camadas PHY, MAC, NET, TRANSP e APP. O pacote é transmitido via rádio LoRa para o Gateway Monocanal na frequência de 916.8 MHz.

O Gateway recebe o pacote via o transceptor RFM95, decodifica e encapsula no protocolo Semtech UDP, encaminhando via rede WiFi para o ChirpStack Gateway Bridge na porta UDP 1700. O ChirpStack processa o payload e disponibiliza os dados na interface web administrativa.

1.5 Arquitetura de 5 Camadas do End Device (TpM)

O firmware do End Device implementa uma arquitetura em 5 camadas independentes para organização do payload MoT de uplink:

Camada	Bytes	Função
PHY (Física)	0-3	RSSI, SNR e parâmetros de rádio
MAC	4-7	Frame Counter e controle de acesso
NET (Rede)	8-11	Endereçamento origem/destino
TRANSP (Transporte)	12-15	Contadores de pacote e controle de sessão
APP (Aplicação)	16-35	Dados do sensor LDR, bateria, GPS e botão

2. Instalação de Ferramentas e Ambiente de Software

Esta seção descreve o passo a passo para Instalação e configuração de todas as ferramentas de software necessárias para desenvolvimento, compilação e deployment da infraestrutura LoRaWAN.

2.1 Instalação do Docker e Docker Compose

O ChirpStack v4 executa em containers Docker. Siga os comandos abaixo para Instalação em ambiente Linux (Ubuntu/Debian):

```
# Atualizar repositórios
sudo apt update && sudo apt upgrade -y

# Instalar dependências
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release

# Adicionar chave GPG oficial do Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
```

```
# Adicionar repositório do Docker
echo 'deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable' | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

# Instalar Docker Engine e Docker Compose
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Adicionar usuário ao grupo docker (evita uso de sudo)
sudo usermod -aG docker $USER
newgrp docker
```

Para validar a Instalação, execute:

```
docker --version
# Docker version 24.0.x ou superior

docker compose version
# Docker Compose version v2.x.x ou superior
```

Para Windows com WSL2, instale o Docker Desktop e habilite a integração com WSL2 nas configurações.

2.2 Instalação e Configuração da Arduino IDE

A Arduino IDE é utilizada para compilação e upload dos firmwares do Gateway e End Device. Siga as instruções:

2.2.1 Download e Instalação

Baixe a Arduino IDE 2.x ou 1.8.x do site oficial: <https://www.arduino.cc/en/software>. Para Linux, utilize o AppImage ou execute os comandos:

```
# Arduino IDE 2.x (AppImage)
wget https://downloads.arduino.cc/arduino-ide/arduino-ide_2.3.2_Linux_64bit.AppImage
chmod +x arduino-ide_2.3.2_Linux_64bit.AppImage
./arduino-ide_2.3.2_Linux_64bit.AppImage
```

2.2.2 Adição da URL de Placas ESP32

Abra a Arduino IDE e acesse o menu: Arquivo > Preferências. No campo "URLs Adicionais para Gerenciador de Placas", adicione exatamente a seguinte URL:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package\_esp32\_index.json
```

2.2.3 Instalação do Core ESP32

Acesse: Ferramentas > Placa > Gerenciador de Placas. Pesquise por "esp32" e instale o pacote "esp32 by Espressif Systems" na VERSÃO EXATA 3.3.7.

Selecione a placa: Ferramentas > Placa > ESP32 Arduino > ESP32 Dev Module.

2.3 Instalação das Bibliotecas Arduino

Acesse: Sketch > Incluir Biblioteca > Gerenciar Bibliotecas. Instale as seguintes bibliotecas nas versões especificadas:

Biblioteca	Versão
MCCI LoRaWAN LMIC library by IBM, MCCI	4.1.1
Adafruit NeoPixel by Adafruit	1.9.0
Adafruit SSD1306 by Adafruit	2.5.16
Adafruit GFX Library by Adafruit	1.12.4
Adafruit BusIO by Adafruit	1.17.4
ArduinoJson by Benoit Blanchon	7.4.3
ESP32httpUpdate by tobozo	2.1.145
ESP8266 and ESP32 OLED driver for SSD1306 displays by ThingPulse	4.4.1
HotButton by germanbraun	0.1.1
LoRaWAN_ESP32 by ropg	1.2.0
Streaming by janelia-arduino	5.0.0
Time by Michael Margolis	1.6.1
WiFiManager by tzapu	2.0.16-rc.2
base64 by Densaugeo	1.0.0

3. Network Server - ChirpStack v4 via Docker

O ChirpStack é uma implementação open-source de Network Server LoRaWAN. Neste projeto, é executado localmente via Docker Compose com todos os serviços necessários para operação completa.

3.1 Clonagem do Repositório

```
# Clonar o repositório oficial
git clone https://github.com/chirpstack/chirpstack-docker.git
cd chirpstack-docker

# O diretório contém a estrutura:
# chirpstack-docker/
#   docker-compose.yml          # Definição dos serviços
#   configuration/
#     chirpstack/
#       chirpstack.toml          # Configuração principal
#       region_*.toml            # Configurações regionais
#   chirpstack-gateway-bridge/
#     chirpstack-gateway-bridge.toml
#     chirpstack-gateway-bridge-basicstation-*.toml
#   mosquito/
```



```
#      config/mosquitto.conf  # Broker MQTT
#      postgresql/
#      initdb/001-init.sql    # Inicialização do banco
```

3.2 Configuração do docker-compose.yml

Substitua o conteúdo do arquivo docker-compose.yml pelo seguinte, configurado para a região AU915:

```
version: "3.9"

services:
  chirpstack:
    image: chirpstack/chirpstack:4
    command: -c /etc/chirpstack
    restart: unless-stopped
    volumes:
      - ./configuration/chirpstack:/etc/chirpstack
    depends_on:
      - postgres
      - mosquitto
      - redis
    environment:
      - MQTT_BROKER_HOST=mosquitto
      - REDIS_HOST=redis
      - POSTGRES_HOST=postgres
    ports:
      - "8080:8080"

  chirpstack-gateway-bridge:
    image: chirpstack/chirpstack-gateway-bridge:4
    restart: unless-stopped
    ports:
      - "1700:1700/udp"
    volumes:
      - ./configuration/chirpstack-gateway-bridge:/etc/chirpstack-gateway-bridge
    environment:
      - INTEGRATION__MQTT__EVENT_TOPIC_TEMPLATE=au915_0/gateway/{{ .GatewayID }}/event/{{
.EventType }}
      - INTEGRATION__MQTT__STATE_TOPIC_TEMPLATE=au915_0/gateway/{{ .GatewayID }}/state/{{
.StateType }}
      - INTEGRATION__MQTT__COMMAND_TOPIC_TEMPLATE=au915_0/gateway/{{ .GatewayID }}/command/#
    depends_on:
      - mosquitto

  chirpstack-gateway-bridge-basicstation:
    image: chirpstack/chirpstack-gateway-bridge:4
    restart: unless-stopped
    command: -c
/etc/chirpstack-gateway-bridge/chirpstack-gateway-bridge-basicstation-au915_0.toml
    ports:
      - "3001:3001"
    volumes:
      - ./configuration/chirpstack-gateway-bridge:/etc/chirpstack-gateway-bridge
    depends_on:
      - mosquitto

  chirpstack-rest-api:
    image: chirpstack/chirpstack-rest-api:4
    restart: unless-stopped
```

```

command: --server chirpstack:8080 --bind 0.0.0.0:8090 --insecure
ports:
  - "8090:8090"
depends_on:
  - chirpstack

postgres:
  image: postgres:14-alpine
  restart: unless-stopped
  volumes:
    - ./configuration/postgresql/initdb:/docker-entrypoint-initdb.d
    - postgresqldata:/var/lib/postgresql/data
  environment:
    - POSTGRES_USER=chirpstack
    - POSTGRES_PASSWORD=chirpstack

redis:
  image: redis:7-alpine
  restart: unless-stopped
  command: redis-server --save 300 1 --save 60 100 --appendonly no
  volumes:
    - redisdata:/data

mosquitto:
  image: eclipse-mosquitto:2
  restart: unless-stopped
  ports:
    - "1883:1883"
  volumes:
    - ./configuration/mosquitto/config/:/mosquitto/config/

volumes:
  postgresqldata:
  redisdata:

```

3.3 Configuração Regional AU915

Edite o arquivo `configuration/chirpstack/chirpstack.toml` para habilitar a região AU915:

```

[logging]
  level="info"

[postgresql]
  dsn="postgres://chirpstack:chirpstack@$POSTGRES_HOST/chirpstack?sslmode=disable"

[redis]
  servers=["redis://$REDIS_HOST/"]
  tls_enabled=false

[network]
  net_id="000000"
  enabled_regions=["au915_0"]

[api]
  bind="0.0.0.0:8080"
  secret="you-must-replace-this"

[integration]
  enabled=["mqtt"]

```

```
[integration.mqtt]
server="tcp://$MQTT_BROKER_HOST:1883/"
json=true

[gateway.backend]
type="mqtt"

[gateway.backend.mqtt]
server="tcp://$MQTT_BROKER_HOST:1883/"
event_topic_prefix="au915_0"
command_topic_prefix="au915_0"
clean_session=false
qos=0
keep_alive="30s"
tls_enabled=false
```

Copie o arquivo de região AU915:

```
cp configuration/chirpstack/region_au915_0.toml.example
configuration/chirpstack/region_au915_0.toml
```

3.4 Inicialização dos Serviços

```
# Iniciar todos os serviços em background
docker compose up -d

# Verificar status dos contêineres
docker compose ps

# Visualizar logs
docker compose logs -f chirpstack
```

Acesse a interface web administrativa em: <http://localhost:8080>. As credenciais padrão são: admin, senha: admin. Altere a senha no primeiro acesso.

4. Configuração e Firmware do Gateway Monocanal ESP32

O Gateway Monocanal é baseado no projeto ESP-1ch-Gateway de Maarten Westenberg, adaptado para operação na banda AU915 com o hardware PKLoRaWAN.

4.1 Wiring - Conexão RFM95 ao ESP32 (DevKit v1)

A pinagem segue a diretiva `_PIN_OUT == 4` definida no arquivo `loraModem.h`. Conecte os componentes conforme a tabela:

RFM95	ESP32 DevKit v1	Função
NSS	GPIO 18	Chip Select (SS)
SCK	GPIO 5	Serial Clock

RFM95	ESP32 DevKit v1	Função
MOSI	GPIO 27	Master Out Slave In
MISO	GPIO 19	Master In Slave Out
DIO0	GPIO 26	Interrupcao TX/RX done
DIO1	GPIO 35	Interrupcao CAD detect
DIO2	GPIO 34	(Reserva)
RST	GPIO 14	Reset do rádio
3.3V	3.3V	Alimentação
GND	GND	Terra

IMPORTANTE: O ESP32 opera a 3.3V. Não alimente o RFM95 com 5V sob risco de danificar o rádio.

4.2 Configuração do Firmware do Gateway

O firmware do gateway consiste em múltiplos arquivos .ino e cabeçalhos .h. Configure cada arquivo conforme descrito nas subseções seguintes.

4.2.1 Arquivo loraModem.h - Pinagem e Frequências

Verifique se a seção PIN OUT == 4 está corretamente definida para o ESP32 DevKit v1 com o modulo PKLoRaWAN:

```
#elif _PIN_OUT==4
// For ESP32/TTGO based board - placa PKLoRaWAN
// SCK == GPIO5/ PIN5
// SS == GPIO18/ PIN18 CS
// MISO == GPIO19/ PIN19
// MOSI == GPIO27/ PIN27
// RST == GPIO14/ PIN14
struct pins {
    uint8_t dio0=26;    // GPIO26/ Dio0
    uint8_t dio1=35;    // GPIO35/ CAD
    uint8_t dio2=34;    // GPIO34/ (reserva)
    uint8_t ss=18;      // GPIO18/ CS
    uint8_t rst=14;     // GPIO14/ Reset
} pins;
#define SCK 5
#define MISO 19
#define MOSI 27
#define RST 14
#define SS 18
```

Verifique a seção AU915_928 para as frequências corretas:

```
#elif defined(AU915_928)
vector freqs [] = {
    { 916800000, 125, 7, 10, 916800000, 125, 7, 12}, // Canal 0 - 916.8 MHz (primário)
    { 917000000, 125, 7, 10, 917000000, 125, 7, 12}, // Canal 1 - 917.0 MHz
    { 917200000, 125, 7, 10, 917200000, 125, 7, 12}, // Canal 2 - 917.2 MHz
    { 917400000, 125, 7, 10, 917400000, 125, 7, 12}, // Canal 3 - 917.4 MHz
    { 917600000, 125, 7, 10, 917600000, 125, 7, 12}, // Canal 4 - 917.6 MHz
    { 917800000, 125, 7, 10, 917800000, 125, 7, 12}, // Canal 5 - 917.8 MHz
    { 918000000, 125, 7, 10, 918000000, 125, 7, 12}, // Canal 6 - 918.0 MHz
```

```
{ 918200000, 125, 7, 10, 918200000, 125, 7, 12}, // Canal 7 - 918.2 MHz
{ 917500000, 500, 8, 8, 0, 0, 0, 0} // Canal 8 - 917.5 MHz BW500
};
```

4.2.2 Arquivo configGway.h - Configurações Principais

Este arquivo contém todas as definições de compilação do gateway. Configure os seguintes parâmetros:

```
// Linha 91: Habilitar banda AU915
#define AU915_928 1

// Linha 105: Habilitar protocolo Semtech UDP
#define _UDPROUTER 1

// Linha 121: Spreading Factor padrão (SF7)
#define _SPREADING SF7

// Linha 131: Habilitar Channel Activity Detection
#define _CAD 1

// Linha 171: Selecionar pinagem do ESP32 PKLoRaWAN
#define _PIN_OUT 4

// Linha 186: Modo strict monocanal
#define _STRICT_1CH 1

// Linha 222: Desabilitar display OLED
#define _OLED 0

// Linha 380: IP do servidor ChirpStack (altere para o IP da máquina)
#define _TTNSERVER "192.168.1.100"
#define _TTNPORT 1700
```

IMPORTANTE: Substitua o IP 192.168.1.100 pelo endereço IP real da máquina onde o ChirpStack está executando. Descubra o IP com o comando: `ip addr show` ou `ifconfig`.

4.2.3 Arquivo configNode.h - WiFi e Identificação

```
// WiFi definitions - Configure suas redes
struct wpas {
    char login[32];
    char passw[64];
};

wpas wpa[] = {
    {"NOME_DA_REDE_WIFI", "SENHA_DA_REDE_WIFI"},
    {"REDE_SECUNDARIA", "SENHA_SECUNDARIA"}
};

// Gateway Identification
#define _DESCRIPTION "Gateway-PKLoRaWAN-01"
#define _EMAIL "seu-email@example.com"
#define _PLATFORM "ESP32"
#define _LAT -22.820923 // Latitude
#define _LON -47.066795 // Longitude
#define _ALT 625 // Altitude em metros
```

4.2.4 Arquivo _utils.ino - Correção de Tipo de Dado

Localize a linha 475 do arquivo _utils.ino e substitua o tipo da variável de "byte" para "uint8_t":

```
// Linha 475 - ANTES:
// byte packetBuffer[NTP_PACKET_SIZE];

// Linha 475 - DEPOIS:
uint8_t packetBuffer[NTP_PACKET_SIZE];
```

4.2.5 Arquivo ESP_sc_gway_pk.ino - Reconexão WiFi

Verifique se a função de reconexão automática está configurada corretamente na seção setup():

```
// Na funcao setup(), verifique a linha com WiFi.setAutoReconnect:
WiFi.setAutoReconnect(true); // Mantenha esta linha
// NÃO use WiFi.setAutoConnect(true); - esta função pode causar problemas
```

4.3 Upload do Firmware do Gateway

Na Arduino IDE, selecione os parâmetros e faça o upload:

```
// Ferramentas > Placa: ESP32 Dev Module
// Ferramentas > Porta: /dev/ttyUSB0 (Linux) ou COMx (Windows)
// Ferramentas > Upload Speed: 921600
// Ferramentas > CPU Frequency: 240 MHz

// Após o upload, abra o Serial Monitor (115200 baud)
// Mensagens esperadas:
// - "Connecting to WiFi..."
// - IP atribuído
// - "CAD start" (indica que o gateway está escutando)
```

Acesse o IP do gateway no navegador para visualizar o painel web com estatísticas de pacotes recebidos e transmitidos.

5. Configuração e Firmware do End Device ESP32

O End Device é o no sensor responsável pela coleta de dados do ambiente (luminosidade via LDR) e transmissão via LoRaWAN Classe A em modo ABP. O firmware foi desenvolvido seguindo a metodologia TpM (Three-Phase Methodology) com arquitetura em 5 camadas independentes.

5.1 Wiring - Conexões do End Device

Conecte os componentes ao ESP32 DevKit v1 conforme a tabela:

Componente	Pino ESP32	Função
RFM95 NSS	GPIO 18	Chip Select
RFM95 SCK	GPIO 5	Clock SPI

Componente	Pino ESP32	Função
RFM95 MOSI	GPIO 27	Dados SPI
RFM95 MISO	GPIO 19	Dados SPI
RFM95 DIO0	GPIO 26	Interrupcao
RFM95 DIO1	GPIO 35	Interrupcao
RFM95 DIO2	GPIO 34	Interrupcao
RFM95 RST	GPIO 14	Reset
RFM95 3.3V	3.3V	Alimentação
RFM95 GND	GND	Terra
LDR (pino 1)	GPIO 36 (ADC1_CH0)	Entrada analogica
LDR (pino 2)	3.3V	Alimentação
Resistor 10k	GPIO 36 a GND	Pull-down
LED Vermelho (anodo)	GPIO 4	Indicador TX
LED Vermelho (catodo)	GND via 220R	Terra
LED Amarelo (anodo)	GPIO 2	Indicador status
LED Amarelo (catodo)	GND via 220R	Terra
LED Verde (anodo)	GPIO 15	Indicador OK
LED Verde (catodo)	GND via 220R	Terra
Botão	GPIO 39 (VN)	Entrada digital
Botão	GND	Terra

5.2 Configuração da Biblioteca LMIC

5.2.1 Arquivo lmic_project_config.h

Crie o arquivo lmic_project_config.h na MESMA pasta do sketch .ino do End Device. Este arquivo é obrigatório para definir o tipo de rádio, a região e contornar conflitos de compilacao com o ESP32:

```
// =====
// lmic_project_config.h
// Coloque este arquivo na MESMA pasta que seu .ino
// =====

// Tipo de radio: RFM95 usa SX1276
#define CFG_sx1276_radio

// Regiao: AU915 (Brasil / Australia 915 MHz)
#define CFG_au915

// CORREÇÃO CRÍTICA para ESP32: renomeia hal_init para evitar
// conflito com a SDK do ESP32 (libpp.a)
#define hal_init LMIC_hal_init

// Suprime warning de pinmap explicito
#define ARDUINO_LMIC_PROJECT_CONFIG_H_SUPPRESS_WARNING
```

```
// Opcional: reduz uso de flash para dispositivos Classe A
// #define DISABLE_BEACONS
// #define DISABLE_PING
```

5.2.2 Correção no Arquivo hal.cpp da Biblioteca LMIC

Esta etapa é CRÍTICA para evitar erro de "multiple definition of hal_init" durante a compilação.

Localize o arquivo hal.cpp dentro da biblioteca LMIC instalada:

```
// Caminho típico da biblioteca:
// Linux: ~/Arduino/libraries/MCCI_LoRaWAN_LMIC_library/src/hal/
// Windows: C:\Users\<usuario>\Documents\Arduino\libraries\MCCI_LoRaWAN_LMIC_library\src\hal\
//
// Abra o arquivo hal.cpp e localize a função:
int hal_init (void) { ... }
//
// Substitua o nome da função para:
int LMIC_hal_init (void) { ... }
//
// Faça a mesma alteração na declaração no arquivo hal.h
```

5.3 Configuração do Canal AU915 no Firmware

O arquivo 1_PHY.ino configura a camada física do End Device para operar exclusivamente no canal 8 (916.8 MHz) da banda AU915:

```
void configura_camada_fisica() {
    // 1. Desativa todos os 72 canais padrão da especificação
    for (int i = 0; i < 72; i++) {
        LMIC_disableChannel(i);
    }

    // 2. Habilita APENAS o canal 8 = 916.8 MHz
    LMIC_enableChannel(8);

    // 3. Define Data Rate (SF7, BW125) e potencia (14 dBm)
    LMIC_setDrTxpow(DR_SF7, 14);

    // 4. Desativa ADR para manter SF fixo
    LMIC_setAdrMode(0);

    // 5. Desativa link check
    LMIC_setLinkCheckMode(0);
}
```

5.4 Credenciais ABP

As credenciais LoRaWAN (ABP) são configuradas no arquivo **ESP-END-DEVICE-LORAWAN-ABP.ino**.

Estes valores DEVEM corresponder aos gerados no ChirpStack durante o registro do dispositivo (ver Seção 6):

```
// Device Address - 4 bytes (MSB primeiro)
static const u4_t DEVADDR = 0x01F5128C;

// Network Session Key - 16 bytes (MSB primeiro)
static const u1_t PROGMEM NWKSKKEY[16] = {
```



```

    0x4D, 0xE6, 0xF6, 0x66, 0xB3, 0x29, 0x56, 0x4C,
    0x7D, 0x5F, 0x0B, 0xD1, 0xBD, 0xA3, 0x15, 0x8A
};

// Application Session Key - 16 bytes (MSB primeiro)
static const u1_t PROGMEM APPSKEY[16] = {
    0x13, 0xC3, 0x56, 0x99, 0x62, 0x60, 0x9A,
    0x78, 0x7B, 0x3B, 0x37, 0xCA, 0x3D, 0x7D, 0xDE, 0xD8
};

```

5.5 Arquitetura em Camadas do Payload

O firmware utiliza uma abordagem de 5 camadas para montar o pacote MoT de 36 bytes:

5.5.1 Camada PHY (1_PHY.ino)

Responsável pela configuração do canal, data rate, potência e bytes 0-3 do payload (RSSI e SNR medidos pelo rádio).

5.5.2 Camada MAC (2_MAC.ino)

Gerencia o frame counter (FCntUp) com persistência NVS usando técnica de wear leveling em 100 slots rotativos:

```

// Bytes 4-5: Frame Counter Uplink
Pacote_UL[4] = (fcntUp / 256);
Pacote_UL[5] = (fcntUp % 256);

// A persistência ocorre a cada FCNT_SAVE_EVERY uplinks
// 100 slots rotativos no NVS distribuem o desgaste da flash
// Vida útil estimada: ~10.000.000 uplinks (vs ~100.000 sem wear leveling)

```

5.5.3 Camada NET (3_NET.ino)

Bytes 8-11: Endereçamento de rede (origem/destino).

```

Pacote_UL[8] = GATEWAY_ID; // Destino (Gateway)
Pacote_UL[10] = MY_ID;     // Origem (End Device)

```

5.5.4 Camada TRANSP (4_TRANSP.ino)

Bytes 12-15: Contadores de pacote da sessão atual.

5.5.5 Camada APP (5_APP.ino)

Bytes 16-35: Dados dos sensores e GPS:

Byte(s)	Conteúdo	Tipo
16	Tipo do sensor (44 = LDR)	uint8
17-18	Valor do LDR (12-bit ADC)	uint16
19	Tipo sensor bateria (11)	uint8

Byte(s)	Conteúdo	Tipo
20-21	Tensão bateria x 100	uint16
22	Estado do Botão (0/1)	uint8
23	Tipo sensor GPS (10)	uint8
24-27	Latitude (float x 1.000.000)	int32
28-31	Longitude (float x 1.000.000)	int32
32-35	Altitude (metros)	int32

5.6 Upload do Firmware do End Device

```
// Arduino IDE configurações:
// Ferramentas > Placa: ESP32 Dev Module
// Ferramentas > Porta: selecione a porta USB
// Ferramentas > Partition Scheme: Default 4MB with spiffs
// Ferramentas > CPU Frequency: 240 MHz

// Após upload, abra Serial Monitor (115200 baud)
// Mensagem esperada a cada 15 segundos:
// [TX] Pacote Enviado
// [LMIC] EV_TXCOMPLETE
```

6. Comissionamento no ChirpStack e Validação da Rede

Esta seção descreve o processo de configuração dos dispositivos na interface web do ChirpStack e a validação completa do sistema.

6.1 Criação do Gateway Profile

Antes de registrar o gateway, crie um perfil de gateway:

1. Acesse <http://localhost:8080>
2. Login: admin / admin
3. Menu: Gateways > Gateway Profiles > Add
4. Preencha:
 - Nome: Gateway-Profile-AU915
 - Region: au915_0
 - Channel plan: AU915
5. Clique em Submit

6.2 Registro do Gateway

1. Menu: Gateways > Add
2. Preencha:
 - Name: Gateway-PKLoRaWAN-01
 - Gateway ID: <EUI do gateway exibido no Serial Monitor>

- Gateway profile: Gateway-Profile-AU915
3. Clique em Submit

O Gateway ID (EUI) é exibido no Serial Monitor do gateway durante a inicialização. É composto por 8 bytes hexadecimais no formato: AA:BB:CC:DD:EE:FF:00:11

6.3 Criação do Device Profile

1. Menu: Device Profiles > Add
2. Aba General:
 - Name: Device-Profile-AU915-ABP
 - Region: au915_0
 - MAC version: LoRaWAN 1.0.3
 - Revision: B
3. Aba Join (OTAA/ABP):
 - Selecione: Supports ABP
 - Desmarque: Supports OTAA
4. Clique em Submit

6.4 Registro do End Device

1. Menu: Applications > Add (crie uma aplicação)
2. Dentro da aplicação: Devices > Add
3. Preencha:
 - Name: EndDevice-Sensor-LDR-01
 - Device EUI: <DevEUI do ESP32>
 - Device profile: Device-Profile-AU915-ABP
4. Aba Activation (Keys):
 - Device address: <copie o DEVADDR do firmware>
 - Network session key: <copie o NWKSKEY do firmware>
 - Application session key: <copie o APPSKEY do firmware>
5. Aba Activation (ABP):
 - FCnt up: 0 (ou sincronize com o valor atual)
 - FCnt down: 0
6. Clique em Submit

6.5 Validação do Sistema

Com todos os componentes em operação, verifique os seguintes pontos de validação:

Local	Verificação	Resultado Esperado
Serial Monitor Gateway	Mensagens após boot	"CAD start" + IP atribuído
Serial Monitor End Device	A cada 15 segundos	"[TX] Pacote Enviado" + "EV_TXCOMPLETE"
ChirpStack > Gateways	Status do gateway	Online com pacotes recebidos
ChirpStack > Devices > Events	Eventos de uplink	Payload hexadecimal visível
Navegador (IP Gateway)	Painel web do gateway	Estatísticas RX/TX atualizadas

6.6 Verificação do Payload no ChirpStack

No ChirpStack, acesse: Devices > (seu dispositivo) > Events. Clique em um evento de tipo "up" para visualizar:

- Frame counter (fCnt) incrementando a cada pacote
- Payload em formato hexadecimal (36 bytes)
- RSSI e SNR medidos pelo gateway
- Frequência confirmada: 916800000 Hz

7. Troubleshooting

Problema	Causa Provável	Solução
Gateway não conecta ao ChirpStack	_TTNSERVER com IP errado	Verifique IP com 'ip addr' e atualize em configGway.h
End Device não transmite	Canal errado para a região	Em 1_PHY.ino, verifique LMIC_enableChannel(8)
ChirpStack não recebe uplinks	Região do Gateway Profile diferente da configuração	Use au915_0 em ambos os lados
Frame counter mismatch	ABP sem FCnt sincronizado	Desabilite validação de FCnt no device (para testes)
Placa não compila	Faltou lmic_project_config.h	Coloque o arquivo na mesma pasta do .ino
Erro hal_init conflito	SDK ESP32 vs LMIC	Redefina #define hal_init LMIC_hal_init
Payload com zeros	Variáveis não inicializadas	Verifique inicialização do array Pacote_UL
LDR leitura errática	Resistor pull-down ausente	Verifique resistor 10k entre GPIO36 e GND
WiFi desconecta	Sinal fraco ou interferência	Use WiFi.setAutoReconnect(true)

7.1 Comandos Úteis de Diagnóstico

```
# Verificar logs do ChirpStack
docker compose logs -f chirpstack

# Verificar se a porta UDP 1700 está aberta
sudo netstat -ulnp | grep 1700

# Testar conectividade MQTT
mosquitto_sub -h localhost -t "au915_0/gateway/+/event/+" -v

# Verificar IP da máquina
ip addr show | grep inet
```

PKLoRaWAN

Infraestrutura LoRaWAN Completa

ESP32 | RFM95 | ChirpStack | AU915 | Classe A | ABP

Documento gerado em junho de 2026